

COMPENDIO COMPLETO D'ESAME: VR, AR & SIMULAZIONE FISICA

Spiegazioni ad alta efficienza per l'orale | Formattazione pulita ed elegante

1. FONDAMENTI DI VR / AR & GRAFICA 3D

Pagina 1-3

Milgram Continuum, Architettura VR & Problemi Sensoriali

- **Milgram Continuum:** Va da *Real Environment* a *Virtual Environment* (VR). La **Mixed Reality (MR)** comprende sia l'**AR** (sovrappone il virtuale sul reale) sia l'**Augmented Virtuality (AV)** (integra elementi reali in un mondo virtuale).
- **Architettura VR:** Formata da **VR Engine** (elabora grafica, audio, fisica e aptica), **Software/Databases** (mesh, materiali, logica) e **I/O Devices** (tracciamento input e visori/aptica output per l'Utente).
- **Target 90 FPS:** Mantenere un framerate di almeno 90 FPS è fondamentale: sotto tale soglia aumenta la latenza sensoriale generando mal di testa e disorientamento.
- **Propriocezione:** Capacità fisiologica del corpo di percepire la posizione dei propri arti e lo sforzo muscolare senza l'ausilio della vista.
- **Cyber-Sickness:** Chinetosi causata dall'**incongruenza sensoriale** (es. la vista indica movimento mentre il sistema vestibolare nell'orecchio interno percepisce staticità). Causata da latenza (lag), ritardi di tracking, o campo visivo (FOV) errato.

Pagina 4-7

Matematica 3D, Quaternioni & Interpolazione

- **Coordinate Omogenee:** Un punto 3D $P = (x, y, z)$ diventa un vettore 4D $P = [x, y, z, 1]^T$. Consente di rappresentare traslazioni, rotazioni e scalature tramite moltiplicazioni di matrici 4×4 .
- **Composizione:** Si applica da destra a sinistra. Per ruotare attorno a un punto P : $M = T^{-1} \cdot R \cdot T$ (trasli all'origine T , ruoti R , trasli indietro T^{-1}).
- **LERP (Linear Interpolation):** $L(t) = (1-t)P + tQ = P + t(Q-P)$ per trovare punti intermedi lungo un segmento retto al variare di $t \in [0, 1]$.
- **Quaternioni:** Vettori 4D $q = q_0 + i q_1 + j q_2 + k q_3$ (1 parte reale, 3 immaginarie).
 - **Quaternione Unitario** ($|q| = 1$): Rappresenta una rotazione di angolo θ attorno a un asse unitario a :
$$q = [\cos(\theta/2), a \cdot \sin(\theta/2)]$$
 - **SLERP (Spherical Linear Interpolation):** Interpolazione ad arco costante su un'ipersfera:
$$\text{Slerp}(t, a, b) = [\sin((1-t)\theta) / \sin(\theta)] \cdot a + [\sin(t\theta) / \sin(\theta)] \cdot b$$

FOCUS PAGINA 15

Realtà Aumentata: Spiegazione a Parole dei Flussi HMD Ottico vs HMD Video

- **1. HMD a Visione Ottica Diretta (Optical See-Through):**

L'utente osserva il mondo reale direttamente attraverso le lenti e i combinatori ottici trasparenti situati davanti agli occhi.

Flusso dell'architettura descritto a parole:

Un sensore di tracciamento (**Head Tracker**) misura la posizione e la rotazione della testa (*Head Locations*) e invia i dati al generatore di scene (**Scene Generator**). Quest'ultimo renderizza e genera le immagini grafiche tridimensionali (*Graphic Images*), mandandole ai piccoli monitor interni. I monitor proiettano la grafica direttamente sui **combinatori ottici (Optical Combiners)**. Qui la luce emessa dai monitor si sovrappone fisicamente alla luce naturale proveniente dall'ambiente reale (**Real World**), arrivando infine all'occhio dell'utente (**User's Eye**).

- **Pro:** Risoluzione reale nativa perfetta; zero ritardo (lag) sulla visione del mondo fisico.
- **Contro:** Perde luce, rendendo gli oggetti virtuali **semitrasparenti**; è impossibile riprodurre il nero pieno o nascondere perfettamente la grafica dietro ad oggetti reali.

• 2. Visione Basata su Video (Video See-Through):

Il visore è fisicamente opaco; l'utente non vede direttamente l'ambiente, ma ne osserva la ripresa video combinata con la grafica 3D.

Flusso dell'architettura descritto a parole:

Una o più telecamere esterne (**Video Cameras**) riprendono il mondo reale (*Video of Real World*). Contestualmente, l'**Head Tracker** invia le posizioni della testa (*Head Locations*) allo **Scene Generator**, che genera l'immagine virtuale 3D (*Graphic Images*). Entrambi i flussi (il video reale e la grafica virtuale) vengono inviati al **Video Compositor** (un mixer video hardware/software). Il Video Compositor sovrappone, unisce ed elabora le due sorgenti in un unico flusso video combinato (*Combined Video*), il quale viene mostrato sui monitor interni di fronte agli occhi dell'utente (**User's Eye**).

- **Pro:** Controllo totale sui pixel. Consente di compensare il ritardo, applicare occlusioni perfette (oggetti virtuali dietro a quelli reali), creare il nero opaco o sostituire del tutto il reale.
- **Contro:** Introduce un piccolo ritardo/latenza (lag) tra il movimento della testa e il flusso video visualizzato.

2. INTERAZIONE & APTICA

Pagina 8-12

Interazione 3D, Dispositivi Aptici & Virtual Coupling

• Selezione Isomorfica vs Non-Isomorfica:

- *Isomorfica:* Mappatura 1:1 rigorosa e naturale con la mano fisica dell'utente.
- *Non-Isomorfica:* Metodo artificiale che amplifica le capacità umane (es. **Go-Go Technique**, applica una funzione non lineare per estendere la lunghezza del braccio e raggiungere oggetti lontani).

• **Aptica (Tattile vs Cinestetico):** Il feedback *tattile* stimola la pelle (vibrazioni/texture); il feedback *cinestetico* agisce su muscoli/articolazioni fornendo forze reali.

• **Dispositivi ad Impedenza:** Leggono la posizione/velocità dall'utente (input) e restituiscono in output una forza calcolata sulla penetrazione.

• **Virtual Coupling:** Collega il punto di contatto reale al "proxy virtuale" tramite una molla e uno smorzatore virtuali:

$$\mathbf{F} = \mathbf{k}_L \cdot \mathbf{d} + \delta_L \cdot \mathbf{v}$$

(dove k_L è la rigidità, d l'offset/penetrazione, δ_L lo smorzamento e v la velocità relativa). Evita compenetrazioni ed esplosioni numeriche.

• **Frequenza Loop Aptico:** Deve girare imperativamente a **1 kHz (1000 Hz)** per garantire stabilità sensoriale ed evitare vibrazioni.

3. COLLISION DETECTION & BOUNDING VOLUMES

FOCUS BOUNDING VOLUMES

Volumi di Ingombro, Trade-Off & Test di Intersezione

• **Concetto di Bounding Volume (BV):** Forma geometrica semplice che racchiude un oggetto 3D complesso. Permette di effettuare rapidi test di scarto (*rejection test*) ed evitare calcoli costosi su migliaia di poligoni.

• **Gerarchia del Trade-Off (Aderenza vs Velocità Computazionale):**

$$\text{Sphere} < \text{AABB} < \text{OBB} < \text{8-DOP} < \text{Convex Hull}$$

- **Sphere / AABB:** Test ultra-veloci e memoria minima, ma aderenza scarsa (molti falsi positivi).
- **OBB / Convex Hull:** Aderenza (*bound*) molto stretta all'oggetto reale, ma test d'intersezione più complessi e costosi.
- **Test AABB-AABB:** Due *Axis-Aligned Bounding Boxes* si intersecano se e solo se si sovrappongono contemporaneamente su tutti e tre gli assi (x, y, z).
- **Test OBB-OBB e Teorema dell'Asse Separatore (SAT):** Due *Oriented Bounding Boxes* sono disgiunti se esiste almeno un asse lungo il quale le loro proiezioni non si sovrappongono. In 3D occorre testare fino a **15 assi separatori**: 3 assi locali di A, 3 assi locali di B e 9 assi dati dal prodotto vettoriale tra gli spigoli di A e B ($\mathbf{u}_i \times \mathbf{v}_j$). Condizione: $|\mathbf{T} \cdot \mathbf{L}| > r_A + r_B$.

- **Algoritmo V-Clip & Regioni di Voronoi:** Suddivide lo spazio esterno a un poliedro convesso in *Regioni di Voronoi* associate a vertici, spigoli o facce. L'algoritmo V-Clip aggiorna iterativamente le coppie di caratteristiche tra due poliedri per trovare i punti più vicini o confermare la penetrazione.
- **Bounding Volume Hierarchies (BVH):** Organizza i BV in alberi (solitamente binari) riducendo la complessità da $O(n^2)$ a $O(\log n)$. Se il nodo padre non viene intersecato, l'intero sottoalbero viene potato (*culling*).
- **Partizionamento Spaziale (Octree vs BSP Tree):**
 - **Octree:** Suddivide lo spazio 3D in 8 sotto-cubi uguali ad ogni livello. Flessibile per oggetti dinamici.
 - **BSP Tree (Binary Space Partitioning):** Usa iperpiani di taglio con orientamento arbitrario. Molto lento da costruire; ideale per **geometria statica** (salva i poligoni solo nelle foglie).

4. FISICA DI BASE & DINAMICA DEL CORPO RIGIDO

FOCUS PAGINA 57-59

Fisica di Base: Cinematica, Newton, Forze, Molle e Smorzatori

- **Cinematica Differenziale:** Velocità $\mathbf{v}(t) = \dot{\mathbf{x}}(t) = d\mathbf{x}/dt$; Accelerazione $\mathbf{a}(t) = \dot{\mathbf{v}}(t) = \ddot{\mathbf{x}}(t) = d\mathbf{v}/dt$.
- **Causa del Moto:** Nei motori fisici il movimento è causato dalle **Forze** (non direttamente dalle velocità).
 - **1^a Legge di Newton:** Se $\mathbf{F}_{net} = 0$, il corpo mantiene il suo stato di quiete o di moto rettilineo uniforme.
 - **2^a Legge di Newton:** $\mathbf{F}_{net} = m \cdot \mathbf{a} = m \cdot \ddot{\mathbf{x}}$. L'accelerazione è direttamente proporzionale alla forza e inversamente proporzionale alla massa.
 - **3^a Legge di Newton:** Ad ogni azione corrisponde una reazione uguale e contraria.
- **Particella (Spazio delle Fasi):** Oggetto dotato di massa m , posizione $\mathbf{x}$ e velocità $\mathbf{v}$, senza estensione spaziale. Lo stato a 6D è $\{x_x, x_y, x_z, v_x, v_y, v_z\}$. Le equazioni differenziali ordinarie (ODE) sono: $\dot{\mathbf{x}} = \mathbf{v}$ e $\dot{\mathbf{v}} = \mathbf{F}/m$.
- **Modelli di Forza Fondamentali:**
 - **Gravità:** $\mathbf{F}_g = m \cdot \mathbf{g}$ dove $g = 9.81 \text{ m/s}^2$.
 - **Attrito Statico vs Cinetico:** $F_s \leq \mu_s F_N$ (impedisce il moto finché la forza tangenziale non supera la soglia); $F_k = \mu_k F_N$ (frena l'oggetto durante lo scorrimento).
 - **Molla (Legge di Hooke):** $F_s = -K(l - l_0)$, dove K è la rigidità (N/m), l la lunghezza attuale e l_0 la lunghezza a riposo. Oppone resistenza all'allungamento o compressione.
 - **Smorzatore (Damper):** $F_d = -C(\mathbf{v}_A - \mathbf{v}_B)$, dove C è il coefficiente di smorzamento e $(\mathbf{v}_A - \mathbf{v}_B)$ la velocità relativa. Rallenta le oscillazioni dissipando energia.

FOCUS PAGINA 64

Quaternions vs Rotation Matrices (Dinamica dei Corpi Rigidi)

- **Confronto e Problema della Deriva Numerica:**

Per la simulazione di corpi rigidi, i **quaternioni** sono nettamente superiori alle matrici di rotazione 3×3 :

- Le matrici usano **9 parametri** (alta ridondanza), e le continue moltiplicazioni ad ogni frame introducono errori di arrotondamento che deformano la matrice (*deriva numerica*).
- I quaternioni usano solo **4 parametri**, riducendo la ridondanza e rendendo banalissima la rinormalizzazione ($|q| = 1$).
- **Rappresentazione Rotazione:** Un angolo θ attorno all'asse unitario $\mathbf{u}$: $\mathbf{q} = [\cos(\theta/2), \sin(\theta/2)\mathbf{u}]$.
- **Derivata Temporale del Quaternione:** Regola la variazione dell'orientamento in base alla velocità angolare $\boldsymbol{\omega}(t)$:

$$\dot{\mathbf{q}}(t) = \frac{1}{2} \cdot \boldsymbol{\omega}(t) \cdot \mathbf{q}(t)$$

(Dove $\boldsymbol{\omega}(t)\mathbf{q}(t)$ indica la moltiplicazione tra i quaternioni $[0, \boldsymbol{\omega}(t)]$ e $\mathbf{q}(t)$).

FOCUS PAGINA 65

Unconstrained Rigid Body Physics: Spiegazione Dettagliata del Frame Loop

- **Stato del Corpo Rigido:** $\mathbf{X}(t) = [\mathbf{x}(t), \mathbf{R}(t), \mathbf{P}(t), \mathbf{L}(t)]^T$ (Posizione centro di massa, Matrice di rotazione, Quantità di moto lineare $\mathbf{P} = m\mathbf{v}$, Momento angolare $\mathbf{L} = \mathbf{I}\boldsymbol{\omega}$).

• Descrizione Testuale del Frame Loop (Step-by-Step):

Fase 1: Pre-Calcolo e Inizializzazione (Eseguita una sola volta)

Si calcola la massa totale del corpo sommando le masse dei punti componenti ($M = \sum m_i$) e il tensore d'inerzia nello spazio locale dell'oggetto (I_{body}). Si definiscono la posizione e la velocità iniziali del centro di massa (x_{CM} , v_{CM}), la matrice di rotazione iniziale R e la velocità angolare ω . Infine si determina il tensore d'inerzia nello spazio mondo ($I = R \cdot I_{body} \cdot R^T$) e il momento angolare iniziale ($L = I \cdot \omega$).

Fase 2: Calcolo ad ogni Frame di Simulazione (Ciclo continuo)

1. *Somma delle forze e momenti*: Si sommano tutte le forze esterne applicate ($F = \sum f_i$) e si calcolano i momenti torcenti totali rispetto al centro di massa ($\tau = \sum r_i \times f_i$).
2. *Integrazione di Eulero per la traslazione*: Si aggiorna la posizione ($x_{CM} \leftarrow x_{CM} + \Delta t \cdot v_{CM}$) e la velocità lineare ($v_{CM} \leftarrow v_{CM} + \Delta t \cdot F/M$).
3. *Aggiornamento della rotazione via Quaternioni*: Si converte la matrice di rotazione R nel quaternione q . Si calcola il nuovo quaternione integrando la velocità angolare ($q \leftarrow q + \Delta t \cdot \frac{1}{2} \omega q$). Si riconverte il quaternione aggiornato nella matrice di rotazione R e si ricalcola il tensore d'inerzia aggiornato nel mondo ($I = R \cdot I_{body} \cdot R^T$).
4. *Aggiornamento della dinamica angolare*: Si aggiorna il momento angolare ($L \leftarrow L + \Delta t \cdot \tau$) e infine si ricava la nuova velocità angolare per il frame successivo ($\omega = I^{-1} \cdot L$).

5. MOTORE FISICO BULLET & DINAMICA AVANZATA

FOCUS PAGINA 67

Pipeline della Libreria Bullet (Spiegazione a Parole del Flusso)

- **Architettura Principale**: Rappresentata da un gestore di alto livello chiamato `btDiscreteDynamicsWorld` che coordina la simulazione ad ogni fotogramma.
- **Descrizione a Parole del Flusso Dati della Pipeline**:

Ad ogni passo di simulazione, Bullet elabora in parallelo i dati di collisione (*Collision Data*) e i dati dinamici (*Dynamics Data*). La pipeline inizia applicando la gravità e predicendo le nuove trasformazioni degli oggetti (**Predict Transforms**). Subito dopo viene eseguita la rilevamento delle collisioni rapida (**Broadphase CD**), che usa gli AABB di ciascuna forma per scartare gli oggetti lontani e identificare le coppie sovrapposte (*Overlapping Pairs*). Le coppie identificate passano alla fase di rilevamento dettagliata (**Narrowphase CD**), che calcola i punti esatti e le normali di contatto (*Contact Points*). Infine, il solutore dei vincoli (**Solve Constraints**) applica gli impulsi fisici per correggere le velocità, posizioni e contatti; l'integratore (**Integrate Position**) calcola le nuove posizioni e le comunica alla grafica tramite la funzione `setWorldTransform`.

FOCUS PAGINA 68

Bullet Physics Library: Simulation Step & Sincronizzazione

- **Passo di Tempo Fisso Interno (60 Hz)**: Per garantire la massima stabilità matematica, Bullet calcola la fisica internamente ad una frequenza costante fissa di **60 Hz** (`fixedTimeStep = 1/60s = 0.01666s`).
- **Sottopassi (maxSubSteps)**: Quando viene chiamata la funzione:

```
dynamicsWorld->stepSimulation(timeStep, maxSubSteps, fixedTimeStep);
```

Bullet prende il tempo trascorso dall'ultimo frame grafico (`timeStep`) e lo suddivide in un numero di sotto-passi interni da 60 Hz (fino ad un massimo di `maxSubSteps`). Questo evita che la fisica vari se il framerate del PC sale o scende.

FOCUS PAGINA 69

Collision Filtering & Classi Base in Bullet

- **Margine di Collisione (Collision Margin)**: Bullet espande le forme geometriche di un piccolo margine (default `0.04 m = 4 cm`). Serve a prevenire la penetrazione visiva tra gli oggetti e rende i calcoli dei punti di contatto enormemente più veloci e stabili.

- **I 3 Metodi di Filtraggio delle Collisioni:**

1. **Masks (Maschere di bit):** Assegna livelli/layer agli oggetti per definire chi può collidere con chi (es. i proiettili ignorano la nave che li spara).
2. **Callback del filtraggio in fase preliminare:** Controlli custom eseguiti nella Broadphase prima di calcolare le collisioni.
3. **Nearcallbacks:** Callback personalizzate eseguite nella Narrowphase (fase ristretta di dettaglio).

- **Gerarchia delle Classi Fondamentali:**

La classe base è `btCollisionObject`, che memorizza la trasformazione nel mondo, la forma 3D di collisione e le proprietà di attrito. Da essa eredita direttamente `btRigidBody`, che vi aggiunge la massa (se maggiore di zero l'oggetto si muove), le velocità lineare e angolare, il tensore d'inerzia locale e la gestione delle forze fisiche.

FOCUS DA PAGINA 70 IN POI

Dinamica del Corpo Rigido Avanzata: Tipi di Corpi, Vincoli e Motion Capture

- **I 3 Tipi di Corpi Rigidi in Bullet:**

- **Corpi Dinamici (Dynamic):** Massa > 0. Rispondono a forze, gravità, impulsi e collisioni. Posizione aggiornata ogni frame.
- **Corpi Statici (Static):** Massa = 0. Non si muovono mai (es. pavimento, pareti), ma gli altri corpi vi rimbalzano contro.
- **Corpi Cinematici (Kinematic):** Massa = 0. Possono essere animati via codice dall'utente. Possono spingere i corpi dinamici ma non reagiscono alle forze.

- **Disattivazione degli Oggetti (Deactivation/Sleeping):** Se le velocità lineare ed angolare di un corpo scendono sotto una soglia minima, Bullet mette l'oggetto in "sleep" saltando l'integrazione numerica per risparmiare CPU. Si risveglia se urtato.

- **Vincoli tra Corpi Rigidi (`btTypedConstraint`):**

- **Point-to-Point (`btPoint2PointConstraint`):** Giunto sferico (3 DOF rotazionali libera, 0 traslazionali). Unisce due oggetti in un punto di pivot.
- **Cerniera (`btHingeConstraint`):** Giunto a cerniera (1 DOF rotazionale). Permette la rotazione solo attorno a un asse (es. porte, ruote).
- **Slitta (`btSliderConstraint`):** 1 DOF traslazionale + 1 DOF rotazionale lungo l'asse X locale.

- **Sincronizzazione Grafica (`btMotionState`):** Interfaccia che permette a Bullet di passare la matrice di trasformazione aggiornata al motore grafico (es. OpenGL, Vulkan) interpolando le posizioni per un rendering fluido.

- **Motion Capture (MoCap Technologies):**

- **Inside-In:** Sensori e marker entrambi sul corpo dell'utente (es. tute con esoscheletri o suite inerziali IMU).
- **Inside-Out:** Sensori sul corpo/visore, marker fissi nell'ambiente (es. telecamere del visore visibile nell'AR).
- **Outside-In:** Sensori/telecamere fisse nell'ambiente, marker passivi/attivi posizionati sul corpo (es. sistemi OptiTrack con telecamere IR a 850nm).